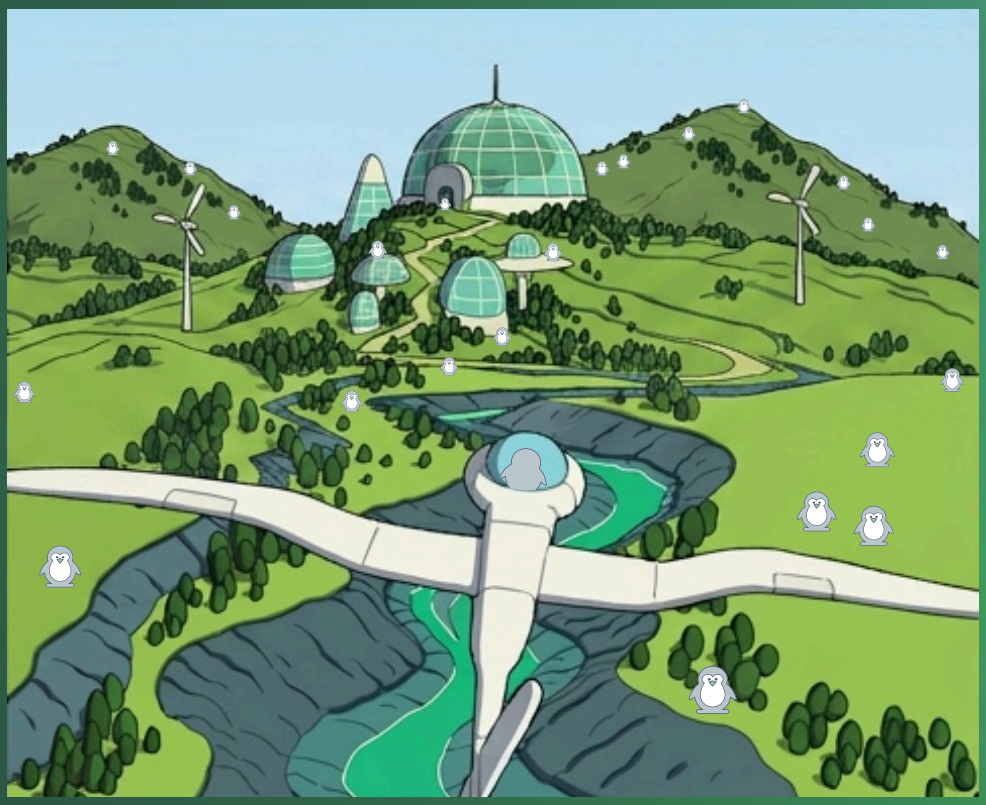
fleet



The IT leader's guide to

Linux device management

About this guide

Linux is no longer a niche concern for enterprise IT. Developers, engineers, scientists, and security researchers have made it their platform of choice. In some organizations, Linux now represents a quarter or more of the total workstation fleet.

That growth creates a management problem. macOS and Windows have mature, well-supported management ecosystems. Linux does not. Many IT teams still rely on scripts, spreadsheets, and tribal knowledge to track and secure their Linux devices. That approach does not scale, and it does not satisfy auditors.

This guide brings together everything IT leaders need to address the Linux management gap: the business case, a framework for defining requirements, an honest look at the technical challenges across provisioning, security, software, and device protection, and a structured way to evaluate the platforms available today.

Fleet publishes this guide. The evaluation criteria and maturity model reflect capabilities we believe any Linux MDM should meet. We encourage you to use them to evaluate any platform, including our competitors.

Contents

Why Linux devices matter now	1
The business case for managing Linux devices	3
Defining your Linux device management needs	6
Automated provisioning for Linux in the enterprise	11
Security baselines for Linux	16
App and certificate management for Linux	19
Protecting the Linux device	22
Controlling your software and your data	25
Choosing the right solution	29

1. Why Linux devices matter now

The Linux project began in 1991 when Linus Torvalds wrote an OS kernel for his own use. He released the code as open source, a model that did not fit mainstream corporate software development of the 1990s.

Steve Ballmer, then CEO of Microsoft, famously called Linux a “cancer”, a direct attack meant to reinforce the idea that Linux had no place in the enterprise.[1] Instead, the comment galvanized a loyal developer community. Linux gained traction first on servers, then in cloud infrastructure, and finally on workstations.

In 2025, Linux crossed 5% of the global desktop market share.[2] That number matters. macOS held roughly 5–10% of the desktop market in the mid-2000s, when it was routinely dismissed as a niche platform fit only for creative work. We know how that turned out. For IT teams, the implication is direct: Linux desktop management is no longer optional.

What is driving Linux desktop adoption?

Windows 10 end-of-life

Windows 10 reached its official end of life in October 2025. Standard support and security fixes ended. Organizations can pay for extended support until 2028, but many are using the transition as an opportunity to reconsider their device strategy. Linux desktop is one of the options gaining traction.

SaaS and web-based workflows

Over the past decade, enterprise software has moved to the cloud. Core business applications are increasingly delivered as SaaS. For developers and engineers, many essential tools are now browser-based or cloud-native. The software compatibility barriers that once blocked Linux desktop adoption are at their lowest level ever.

Cloud-native development

Modern software is commonly deployed on AWS, Azure, or Google Cloud, running in Docker containers and Kubernetes clusters. Those platforms are built on Linux. Developing on Windows often requires an abstraction layer such as Windows Subsystem for Linux (WSL) to emulate a target environment. For teams building and deploying Linux containers in the cloud, using Linux on workstations makes technical sense.

Security and trust

The rise in supply-chain attacks and large-scale malware campaigns has sharpened focus on device visibility and threat detection. For sensitive deployments, open-source systems can offer greater trust, provided devices are properly audited and managed.

Who uses Linux at work?

Linux users are often among an organization's most important contributors: developers, engineers, scientists, researchers, and security practitioners. Stack Overflow's 2025 developer survey found that roughly 28–30% of respondents use Linux as their main operating system for professional work.[3]

Many of these employees deliberately chose Linux. For years, Linux devices lived outside IT control and were treated as special cases, left to end users to self-manage. That era is over. As Linux adoption grows, so does its attractiveness to attackers. Two data points illustrate how quickly the threat landscape has shifted:

- In 2023, the IceFire ransomware family specifically targeted Linux systems, demonstrating that threat actors are actively adapting tools to attack Linux workstations, not just servers.[4]
- Also in 2023, Trend Micro documented a 62% surge in Linux ransomware attack attempts, confirming that Linux is now a priority target for ransomware operators.[5]

Unmanaged Linux devices are a liability. Linux must be managed as a first-class platform, held to the same security and compliance standards as macOS and Windows.

The operational gap

Linux workstations present the same operational challenges as any other device: provisioning and onboarding, software deployment and updates, certificate and configuration management, security baseline enforcement, and compliance reporting.

Most organizations handle these challenges on macOS and Windows using platform-specific management solutions. For Linux, the picture is fragmented. Powerful open-source tools exist: Ansible, Puppet, OpenSCAP, and osquery. Each requires significant investment to learn, integrate, and maintain. Mainstream management platforms like Intune and Workspace ONE offer limited Linux support, leaving the gap unresolved.

The market is catching up. In 2026, organizations need to take Linux management seriously. The rest of this guide explains how.

2. The business case for managing Linux devices

The question facing IT leaders is no longer “should we manage Linux?” It is “what do we gain when we do?” The answer touches cost, compliance, talent retention, and long-term strategy.

Reducing tool sprawl

Most organizations manage macOS devices with one tool, Windows devices with another, and Linux devices with a collection of scripts and spreadsheets, if they manage them at all. Each additional tool carries its own licensing cost, training overhead, and renewal cycle.

A 2024 Forrester survey found that 77% of US technology decision-makers report moderate to extensive technology sprawl, and 63% planned to pursue consolidation.[6]

The cost of sprawl goes beyond licensing fees. Fragmented tooling produces fragmented visibility. When Linux devices live outside the management perimeter, IT cannot consistently answer basic questions across the fleet: Which machines are running which OS versions? Which devices have disk encryption enabled? Where are the unpatched vulnerabilities?

Bringing Linux into the same management framework as macOS and Windows provides a single source of truth. It eliminates parallel processes, reduces context switching for IT staff, and ensures compliance reporting covers the entire fleet.

Strengthening compliance posture

Regulatory frameworks like SOC 2, ISO 27001, HIPAA, and NIST 800-53 do not grant exemptions based on operating system. Every corporate device must be accounted for, patched, encrypted, and policy-compliant. An unmanaged Linux laptop is a gap in the audit trail, and gaps are expensive to explain and harder to remediate after the fact.

Managing Linux devices closes this gap in three ways:

1. **Automated patch management.** Security updates ship on a predictable schedule rather than relying on individual users to run them.
2. **Centralized configuration enforcement.** Disk encryption, screen lock policies, and firewall rules apply across the fleet and can be verified programmatically.
3. **A complete audit trail.** Device inventory, policy status, and remediation history live in the same reporting interface used for macOS and Windows.

When auditors ask how you manage your Linux devices, the answer should be the same as for every other platform: “Here’s the dashboard.”

Earning trust with technical talent

Linux users are developers, engineers, security researchers, and data scientists. These employees chose Linux deliberately. They value transparency and control. They notice, and resist, management software that feels opaque or intrusive.

That resistance creates a real tension. IT needs visibility and control. Linux users want autonomy and transparency. The wrong approach, forcing a Windows-centric management tool onto Linux devices, drives users toward shadow IT.

Research shows that 47% of companies allow employees to access resources on unmanaged devices using credentials alone.[7] Gartner estimates that by 2027, 75% of employees will acquire or create technology outside of IT’s visibility, up from 41% in 2022.[8]

A management strategy that respects this culture reduces friction. That means lightweight agents, open-source tooling where possible, and clear communication about what data is collected and why. When Linux users trust the management tooling, they comply with policy rather than work around it. Shadow IT goes down. Security posture goes up.

Lowering the total cost of ownership

Linux itself does not carry a per-seat licensing fee. But the total cost of ownership for any device goes beyond the OS. It includes management tooling, labor to maintain and troubleshoot, and the cost of incidents caused by improperly configured devices.

Formalizing Linux management reduces cost in three areas:

1. **Less manual labor.** Automation handles patch deployment, configuration enforcement, and software distribution, eliminating the need for custom scripts and hands-on intervention.
2. **Faster incident response.** Managed devices are visible devices. Visible devices receive faster detection and containment, directly reducing breach costs.
3. **Fewer redundant tools.** Linux joins a unified management platform instead of living in a parallel ecosystem of scripts and spreadsheets.

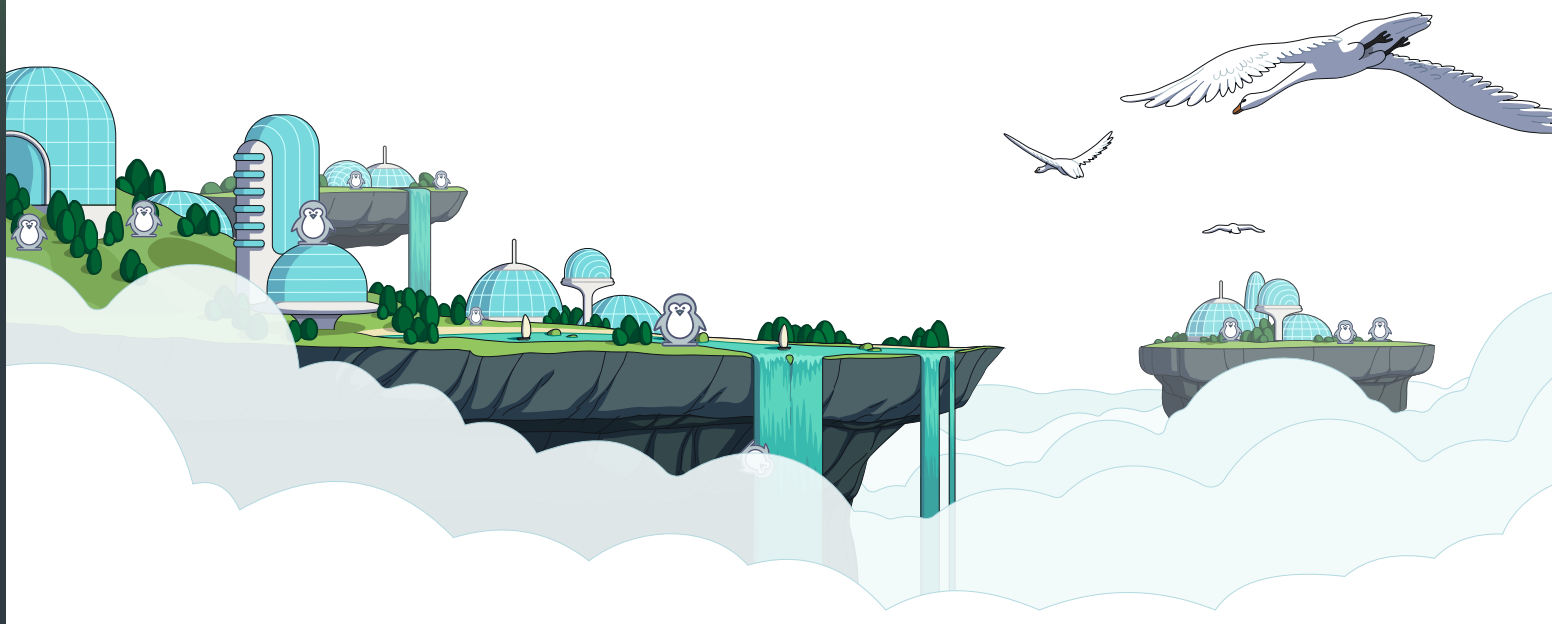
Every hour an IT engineer spends manually auditing a Linux workstation or writing a one-off compliance script is an hour not spent on higher-value work. These costs grow linearly with the fleet. A managed approach scales more efficiently.

Modernizing IT operations

The practices required to manage Linux well, infrastructure as code, version-controlled configurations, automated policy enforcement, and API-driven tooling are the same practices that define modern IT operations.

Organizations that adopt these approaches for Linux often find the benefits extend to their entire fleet. Configuration-as-code practices that start with Linux can apply to macOS and Windows. Automated compliance checks can standardize reporting across all platforms. Shared language between engineering teams and IT operations, version control, pull requests, and peer review reduce friction between traditionally siloed departments.

Managing Linux devices is not an edge case. It is an opportunity to raise the bar for how your organization manages all of its devices.



3. Defining your Linux device management needs

Rushing toward technical implementation without clearly defined needs and success criteria is a mistake. Every business is different. Team dynamics, regulatory requirements, user autonomy expectations, and budget constraints all shape the right answer. Before evaluating any platform, get clear on what you actually need.

Key questions to ask

Do you want to monitor or fully manage Linux devices?

Some teams must give their Linux users a high degree of autonomy. They accept the security trade-offs that come with end users having full system access. Even so, they still need to meet regulatory and compliance goals. They cannot operate without visibility.

If this describes your organization, simple visibility into your Linux devices may be sufficient. Other organizations want more robust management: full control over software on Linux workstations, complex configuration management, and an approved set of packages and configurations for end-users.

What level of configuration management do you need?

Device management is not only a security exercise. Robust configuration management enables an organization to provision workstations for its users, reducing wasted time on setup. Windows teams have had tools like Group Policy for years. Define the level of configuration management your Linux users need before you start evaluating platforms.

Should users install only from an approved software catalog? Can they install and configure their own software? Do you need drift detection and auto-remediation if a device drifts from an acceptable baseline?

Is improving developer velocity a concern?

A robust device management platform can speed up employee onboarding and reduce time spent reconfiguring workstations when software policies change. Many Windows and macOS teams hand a new employee a fully provisioned laptop on day one. With Linux, this is often left to the end user.

The result: team members spend more time setting up their workstations than contributing to their projects. Does your organization want to reduce this burden, or is the status quo acceptable? Self-service portals give users good choices without impinging on their autonomy.

What distributions and software will you support?

One of the core challenges in Linux device management is the wide range of choices available to users. There are hundreds of Linux distributions and dozens of ways to manage configuration. Define which distributions and configurations your IT team is willing to support. Otherwise, you will find it impossible to support everyone's desired workstation.

Engage with your Linux user base. Some will have strong opinions about distributions and can make a compelling case for productivity gains. Others will compromise, as long as they can use Linux. Either way, you need to make deliberate decisions about what you will support and how you will handle exceptions.

How do you want to manage Linux devices?

Most organizations manage devices through manual, UI-driven workflows (sometimes called ClickOps): clicking through web portals to deploy profiles, configure policies, and push software. Some infrastructure teams apply the same infrastructure-as-code and GitOps practices they use for cloud infrastructure to device management.

How you want to manage devices is a foundational decision. It will shape every technology choice that follows and influence where you land on the maturity model described later in this chapter.

What constraints do your users have on system performance?

Linux users often run resource-intensive workloads, such as compiling code, running local containers, and processing large datasets. A management agent that consumes significant CPU or memory on a developer workstation will generate complaints immediately, and those complaints will turn into resistance.

Before evaluating any platform, understand your users' performance expectations. Ask vendors for agent resource benchmarks under realistic workloads, not just idle numbers. Test on the hardware your users actually run.

How will Linux devices connect to your identity infrastructure?

Device management does not exist in isolation from identity. Enrollment, user mapping, software access, and offboarding all depend on your identity provider. If your organization uses Okta, Active Directory, LDAP, or any SCIM-compatible system, your management platform needs to integrate with it.

A platform that manages Linux devices well but requires manual user management defeats much of the automation benefit. It also creates gaps during offboarding, exactly the kind of gap that shows up in a security audit.

How should device data flow into your security toolchain?

Visibility into Linux devices is only useful if that data reaches the people and systems that act on it. A compliance dashboard that lives exclusively inside your device management tool is a dashboard that gets checked inconsistently.

Consider where your team already responds to security events: your SIEM, your ticketing system, your alerting toolchain. Does your management platform integrate with those systems? Can device telemetry trigger automated alerts or feed into existing workflows?

What does your enrollment process need to look like?

Getting from zero managed Linux devices to a fully enrolled fleet is a project in itself. Ask how enrollment works for existing devices already in use. Ask what the experience looks like for the end user: is it self-service, IT-assisted, or fully automated?

If you are migrating from an existing tool or replacing a collection of scripts, ask what that transition path looks like. A difficult enrollment process will stall adoption regardless of how capable the platform is once running.

How does cost scale with your Linux footprint?

Per-device pricing looks different at 200 devices than it does at 2,000. Understand the pricing model before you commit to a platform. Ask what happens to cost at scale and which features are gated behind higher tiers. Some platforms offer generous entry-level pricing but reserve automation and reporting features for enterprise contracts.

Who needs access to Linux device data, and at what level?

A helpdesk technician needs different access than a security analyst. An administrator managing devices in one region may not need visibility into devices in another. Some compliance frameworks require audit logging of who accessed device data and when.

Define your access control requirements before evaluating platforms. A tool that offers only administrator versus read-only access will create friction as your team grows or your compliance requirements become more specific.

The Linux MDM maturity model

Not every organization needs the same level of Linux device management. This model defines four levels, from basic monitoring to full zero-touch provisioning. Your requirements determine which level is right for you. Not every team needs to reach Level 4.

When planning, consider both short- and long-term goals. Start by knowing where you want to land before deciding how far to climb.

Level 1: monitoring and auditing

All teams need some level of device monitoring to ensure compliance with internal and external policies. Robust monitoring provides individual and aggregate metrics to drive intelligent decisions.

Monitoring lets you answer questions like:

- Are all Linux devices running the correct kernel and package versions?
- Are any devices running software versions with known vulnerabilities?
- Has anyone made a change that conflicts with organizational policy, such as an overly permissive sudoer rule or an unauthorized SSH key?

For some organizations, especially small teams with limited Linux footprints, this is the final level on their journey. If you have a handful of Linux workstations, you may not need further management features. Everyone benefits from understanding the state of their environment.

Level 2: security and system baselines

With a solid understanding of your Linux devices, you can begin providing baseline configurations that meet organizational and security policies. Start with something simple, like preventing users from adding local accounts. As you gain experience and user trust, move to more advanced configurations, such as automatically deploying corporate certificates across all Linux workstations.

Many organizations are satisfied stopping here. Providing baseline system management while leaving the rest to highly skilled end users is a legitimate target state.

Level 3: self-service configuration and software

Linux users are technically skilled. Most are comfortable installing and managing their own software. But even advanced users sigh when they have to install a complex package with custom configuration. This is toil, and Linux users often face the brunt of it due to a lack of robust management tooling.

Level 3 is about providing users with access to the tools and software they need to do their jobs. A self-service portal saves hours. It lets users find and install approved software without involving IT for every request.

Level 4: zero-touch provisioning and drift management

Level 4 is full zero-touch provisioning and automated, continuous management of device state. A new Linux device can be powered on and immediately put into service by an end user without any IT involvement. Organizations can onboard new Linux employees the same way they onboard Windows and macOS users.

Level 4 also involves continuous drift management. Linux users often have privileged system access to their workstations. Changes can easily drift from organizational policy. Robust drift management detects these problems and remediates them without involving the end-user.

Use the maturity model as a planning tool, not a finish line. Level 1 is a legitimate long-term target for small teams (e.g. a 10-person start-up). Level 4 makes sense for organizations with large Linux footprints and strict compliance requirements (e.g. a 500-person org with SOC 2 requirements), or engineers who need a consistent, fully provisioned workstation on day one.

4. Automated provisioning for Linux in the enterprise

Linux users love the power and flexibility of their systems. Features like built-in package managers, deep OS customization, native development tooling, and scriptable kernel-level access make Linux the workstation of choice for engineers and developers. But that same power and flexibility widens a critical gap for enterprise IT: Linux “MDM” is a term of art, not a real protocol.

There is no built-in Linux framework for enrolling, provisioning, and configuring desktop computers in the enterprise. No protocol ensures a smooth out-of-box (OOB) experience for end users while guaranteeing device compliance and security. Understanding that gap, and knowing how to close it, is what this chapter is about.

Enabling work

Delivering a frictionless end-user experience when deploying devices to new hires should be a goal every organization aspires to. Smooth employee onboarding says a lot about how an organization functions and communicates. When new hires are confident in the onboarding process, it instills confidence in their role and equips them for success by allowing them to get to work intuitively on day one.

Getting a computer from a manufacturer and setting it up manually, installing software, configuring controls, and securing access to enterprise resources requires meticulous planning, coordination, and significant effort. To deploy manually at scale, many large organizations pay third parties or have dedicated teams to handle this work, given the complexity of delivery logistics and device configuration.

In the early days of Windows PCs, disk imaging became the de facto approach for replicating OS settings during deployments. Windows admins spent enormous amounts of time crafting a ‘golden image’ that was then applied to all newly purchased computers. Some organizations still take this approach, but maintaining and refreshing master images is expensive, fragile, and hard to keep current as software and security requirements change.

Windows has long supported automated provisioning, following Apple’s lead on boosting the use of MDM specifications built into operating systems. These improvements helped organizations modernize their device deployments. Linux has not had the same built-in investment.

Modern device enrollment and provisioning

Apple's Automated Device Enrollment (ADE) and Microsoft's Windows Autopilot protocols have made device enrollment and provisioning far easier than ever before. The core insight behind both systems is the same: tie a physical device to an institutional identity at the point of manufacture or purchase. When the device powers on for the first time, it already knows who it belongs to and what policies to apply.

Much of the improvement to these systems came along at just the right time. During the COVID pandemic, zero-touch device provisioning went from being a "nice-to-have" option to table stakes. IT and client platform engineers could no longer expect new employees to show up at an office, nor could they provide the "white glove" experience of provisioning a computer by hand. Devices had to arrive at an employee's home pre-configured and ready to use on day one.
Linux has no equivalent. There is no MDM specification built into Linux, no Apple Business Manager for Linux, and no central registry for Linux computers comparable to Microsoft Entra. Without that infrastructure, organizations cannot tie a Linux device to institutional ownership at the point of purchase. Every deployment starts from scratch.

The result is that Linux enterprise deployment is often worse than the old days of Windows disk imaging:

- Because of a lack of Linux management options, an organization may allow end users to "manage" devices deployed to them on their own. This may mean they are given management guidelines or security documentation without any form of actual enforcement.
- At the enterprise level, Linux is often used by an organization's most sophisticated end users, meaning a lack of enforcement around Linux device management can be a hollow promise at best or a political football at worst.
- Linux computers are often intended for use by end users with root-level OS access. This means manual creation and deployment of 'golden master' images is practically useless and only functional as a "starting point", not for ongoing management.
- Meanwhile, IT teams asked by security teams to ensure all devices are compliant with regulations and business policy have no control over an important subset of the devices they've been asked to manage.

Why device enrollment and automated provisioning are critical

There are three general reasons why device enrollment into a management solution and automated provisioning are crucial for modern device management.

Device provenance

Properly enrolling devices into a management solution ensures device provenance, establishing ownership from the manufacturer through purchase to the end user. With provenance established, automated provisioning can proceed.

A device moves from an OOB state to a securely configured machine with no IT interaction and minimal end-user friction. A zero-touch provisioning process minimizes the security gap created by shipping new devices directly to end users. Because of this, automated enrollment and provisioning workflows have become indispensable for teams managing devices for remote workers.

End-user onboarding experience

Automated provisioning is one way organizations can deliver the frictionless end-user onboarding experience users desire. For all employees, especially remote workers, automated provisioning ensures new hires are onboarded quickly without requiring help desk calls. They can get straight to business.

A well-provisioned device on day one also sets the tone: it tells the new hire that the organization is competent, prepared, and respects their time.

End user identity

Automated device provisioning establishes the connection between an end user's identity and their assigned device. This means linking local accounts on the device with an enterprise identity provider (IdP) and configuring access in single sign-on (SSO) systems. These are critical provisioning steps for allowing end users to securely access enterprise resources.

Once a user’s identity can be reliably associated with their device, administrators can establish secure access policies based on user attributes and group membership, rather than relying solely on device attributes.

Orchestration approaches for Linux

Because there is no native MDM protocol for Linux, organizations typically rely on orchestration to manage devices at scale. Two broad approaches exist, each with real tradeoffs.

Agent-less: Ansible and similar tools

The agent-less approach, typified by Ansible, uses SSH connections to authenticate and control Linux devices. YAML files and scripts configure devices to match a desired state as defined in an Ansible “playbook”. The intent is declarative state management: define what a device should look like, and Ansible makes it so.

In practice, the approach has a significant limitation. Because Linux end users often have root-level access, any configuration setting can be overridden. Configuration drift becomes a persistent challenge. Changes made by the end user are not reflected back to the Ansible playbook, monitored, or remediated by default. Ansible works well for initial configuration. It is less reliable for continuous enforcement.

There is also an operational overhead: Ansible requires SSH connectivity to each device during execution. For a distributed fleet of remote workers, this can be difficult to guarantee. Playbooks must be run deliberately; they do not run themselves.

Agent-based: Puppet, Chef, and modern management platforms

An agent-based approach installs a lightweight agent on each device. The agent checks in with a management server on a periodic basis, pulls down updated configuration, and can monitor the device for deviations from a known baseline. When drift is detected, the agent remediates it locally.

Tools like Puppet and Chef take this approach, though in practice, they are not full-featured device management solutions. They are strong at configuration enforcement, but they were not designed for the full lifecycle of a corporate device: enrollment, software distribution, compliance reporting, and offboarding.

Modern Linux management platforms extend the agent-based model with the full feature set organizations need. When evaluating a platform, look for:

- Simple agent installation and enrollment
- Script execution on individual devices or groups
- Software installation and patch management
- Device state monitoring and real-time inventory
- Compliance reporting against defined policies
- Automated state remediation when drift is detected

The difference between a configuration management tool and a device management platform lies in the scope of the problems each solves. Configuration management enforces the desired state. Device management covers the full lifecycle: from the moment a device ships through enrollment and provisioning, to daily operation, and eventually to offboarding.

Best practices for automated provisioning on Linux

If your organization purchases computers directly from a large manufacturer such as Dell or Lenovo, or from a third-party reseller such as CDW or SHI, you can typically receive computers preinstalled with Linux. This is often the simplest and best starting point for enterprise Linux deployments. There is no central registry for Linux computers like Apple Business Manager or Microsoft Entra. Starting with a known-good OS installation from a trusted supplier removes one layer of uncertainty.

From there, a modern Linux management platform can take over. A platform with setup experience capabilities can deliver zero-touch provisioning for Linux end users. The device powers on, the user authenticates with their corporate identity, and the device configures itself. Disk encryption enabled. Software installed. Certificates deployed. Security policies applied. No IT hands-on time required.

That same platform links Linux devices to user identity in your IdP, just as you do on macOS and Windows. Enrollment and offboarding become consistent across all platforms in your fleet, not only those that have a built-in MDM protocol.

5. Security baselines for Linux

Linux workstations often host an organization's most valuable assets: source code repositories, architecture plans, and sensitive internal roadmaps. The investment in device management for macOS and Windows is a given. So why is the security posture of Linux workstations so often treated as a special case?

The fundamental challenge is a mismatch between the platform's nature and the enterprise's need for control. Linux distributions, built on a philosophy of power and flexibility, are designed for the power user. Those capabilities, deep OS customization, native development tooling, and root-level access, left unchecked, create a security vacuum.

How other platforms solved this

Windows pioneered centralized control with Group Policy Objects, enabling granular management of password complexity, desktop restrictions, and software deployment from a single console. macOS followed with MDM configuration profiles. Linux has traditionally lacked a native, centralized security enforcement protocol. Scripting and configuration management tools exist, but they tend to be bespoke, leading to fragmented and inconsistent security across a fleet.

The myth of low risk

The perception of low risk often justified the historic lack of scrutiny for Linux security: a smaller installed base meant fewer attackers would target it. That assumption is no longer valid. In 2023, Trend Micro documented a 62% surge in Linux ransomware attack attempts.[5] The time for treating Linux security as an afterthought is over.

What settings to enforce: security baselines

The real challenge for security professionals is not how to execute a security policy. The scripting and configuration tools exist. The challenge is what settings to enforce. Security baselines answer this question.

A security baseline is a consensus-driven, recommended set of configurations applied uniformly across devices to ensure a consistent, hardened security posture. Key examples include:

- **CIS Benchmarks -** Developed through global community consensus by cybersecurity experts, the CIS Benchmarks provide detailed, prescriptive guidance for securing various operating systems, including multiple Linux distributions (Red Hat, Ubuntu, Debian), as well as macOS and Windows.
- **NIST and government frameworks -** The US National Institute of Standards and Technology, the UK's National Cyber Security Center, and similar agencies publish extensive recommendations for securing devices in line with high-security government standards.

These baselines are practical, operational guidance. They cover access controls (password complexity, biometric policies), system configuration (restricting access to sensitive binaries and configuration files), and network configuration (limiting inbound and outbound connections).

Three reasons baselines matter

Simplified auditing and metrics

Baselines provide a consistent yardstick to measure the health and security hygiene of your entire fleet. The ability to audit all workstations, macOS, Windows, and Linux, against a consistent set of metrics establishes a clear objective standard. Tracking and measuring deviations from a well-defined set of requirements is fundamentally easier than building those requirements from the ground up.

The fight against configuration drift

Configuration drift is one of the most persistent security threats. In Linux, most controls are imperative. When an administrator issues a command such as:

```
install corporate root certificate
```

The system executes it once. This is unlike declarative state management, where the system continuously works to maintain a desired configuration. An imperative command does not guarantee persistence. A user with root access, a subsequent script, or even a system update can later remove or replace that certificate.

Adopting a security baseline, combined with tooling that monitors and remediates drift, enables security teams to set a desired state and keep devices in it.

Meeting certification requirements

For many organizations, security standards are not optional. Standards such as SOC 2, ISO/IEC 27001, HIPAA, PCI DSS, and NIST SP 800-53 apply to how an organization processes and protects information, including the security of computing devices. Recognized security baselines allow enterprises to systematically demonstrate compliance and provide auditors with evidence that security practices for all workstations, including Linux, are mature and reliable.

Practical next steps

Adopting a recognized security baseline, such as the CIS Benchmarks for Red Hat, Ubuntu, or Debian, does not require reinventing best practices. The guidance is publicly available. The challenge is consistent enforcement across a distributed fleet.

The right tooling monitors device state against those baselines, flags drift, and remediates automatically. The next chapter covers an adjacent layer: managing the software and certificates on those devices.

6. App and certificate management for Linux

Computers are not intrinsically productive or secure. Their value comes from trusted applications deployed on them and the certificates that underpin connectivity. With a hardened baseline and compliance policies in place, the next layer of enterprise Linux management addresses two challenges: fragmented software distribution and shrinking certificate lifetimes.

The software distribution maze

If you manage Linux workstations, you know there's no single package format or unified app store. Debian-based systems use `apt`, Red Hat-based systems use `dnf`, and SUSE uses Zypper. Then there's Flatpak, Snap, and AppImage on top. Compare this with macOS or Windows, where software distribution is more standardized. On Linux, there's no single ecosystem, so teams that support multiple Linux distributions must manage across all of them simultaneously.

This fragmentation creates real challenges. Each package manager resolves dependencies differently. Each distribution maintains its own repositories with its own release schedules. When your team installs a library from a third-party PPA or compiles software from source, that package often falls outside any centralized tracking. A vulnerability in a shared library can affect dozens of applications, increasing risk if you don't know which devices have it installed.

The software chain of trust

The ability to distribute and install applications securely relies on a chain of trust that extends from the developer to the end user. For modern operating systems, a critical part of this chain is software notarization.

Notarization is a security process where a developer submits their application to the operating system vendor (or a trusted third party) for an automated security scan before it is distributed. The notarization service checks for known malware, signs the app with a secure ticket, and "blesses" the software. This allows the OS to inform the user that the software is free from common threats and comes from an identified source.

Limitations on Linux vs. macOS and Windows

In the enterprise world, macOS and Windows have established mandatory or strongly recommended notarization protocols. With macOS, Apple enforces notarization for apps distributed outside the App Store. Windows provides equivalent code signing that ensures the integrity of executables.

Linux lacks the concept of a universal security notary. Checks are instead relegated to each repository to ensure packages have not been compromised in the supply chain. Some package managers like Debian and Ubuntu rely on manifest data while others like `rpm` rely on individual packages being signed.

Given that Linux was philosophically rooted in openness, the concept of enforcing checks on software before execution may seem out of place. Apple and Microsoft have introduced safety options as a consumer benefit for enforcing checks and warnings about software downloaded and executed from unknown sources. With Linux, these checks are missing and are unlikely to be enforced from a centralized control mechanism like an App Store given the distributed and intentionally open nature of the platform.

The gap in software security based on the fragmentation of Linux software repositories means that IT administrators must enforce more rigorous checks on software installed on Linux devices. With fewer policy controls over software distribution, the need for increased vigilance is critical.

Patching at the speed of threats

Fragmentation becomes a serious problem when threats move fast. The XZ Utils backdoor[9] detected in early 2024 showed how quickly a supply chain compromise can escalate. A malicious contributor spent years gaining trust in a widely used compression library, then inserted code that could have enabled `ssh` compromise on affected systems. The issue was caught before it reached most stable distributions, but the incident exposed a hard truth: when a widely-used, critical Linux package is compromised, the time between disclosure and exploitation can be very short. Sometimes only hours.

That timeline doesn't leave room for manual patching. If your team of two or three engineers manages hundreds of Linux workstations, you can't `ssh` into each machine, verify the installed version, and deploy an update before the window closes. Even with tools like Ansible or Puppet, the process assumes you already know which machines are affected, and that assumption often fails.

Without visibility into installed packages, you can't assess exposure or prioritize remediation. Package names and version strings differ between `apt`, `dnf`, and Zypper for the same upstream project, making it harder to correlate disclosures with what's installed. The gap between "patch available" and "patch deployed" is where attackers operate.

Closing that gap requires knowing what's installed, where, and how to update it before the next disclosure drops.

The certificate lifecycle challenge

Software isn't the only OS asset that requires visibility. Certificates are meant to be invisible unless something is wrong. You don't notice them until Wi-Fi stops connecting, VPN tunnels drop, or internal services reject HTTPS requests. Behind the scenes, certificates form the trust chain that holds enterprise connectivity together: 802.1X authentication for Wi-Fi, mutual TLS for VPNs, HTTPS for web services, and client certificates for identity verification.

That trust chain requires maintenance. The CA/Browser Forum has approved a schedule[10] to reduce maximum public TLS certificate validity from 398 days to 47 days by March 2029. Multi-year certificate lifetimes and manual renewal processes are largely over. As validity windows shrink, automation becomes essential. Protocols like ACME (Automated Certificate Management Environment) can handle renewal, though they're often deployed centrally on gateways or proxies rather than on every device.

On Linux, certificate management adds complexity that varies by distribution. Red Hat-based and Debian-based systems store certificates in different locations and use different commands to process them. Browsers like Firefox often maintain their own certificate stores, separate from the system trust store. A certificate you deploy to the OS may not be recognized by the browser, and vice versa.

While distributing certificates with simple scripts is often adequate for basic deployments, the lack of a centralized certificate store means that administrators need to rely on automated tools. These tools must ensure certificates are injected into the relevant stores on Linux workstations: from browsers, to Wi-Fi and various VPN applications.

Starting now with visibility into what's deployed and when it expires gives you the foundation to scale certificate management before additional browser-enforcement changes further compress renewal windows.

7. Protecting the Linux device

Cloud security is important, but so is the data sitting on a local device. Developer workstations accumulate sensitive material in every corner of the filesystem: cached credentials, SSH keys, API tokens in environment files, and proprietary source code in local repositories. If an attacker gains access to one of these computers, they inherit everything within a privileged end-user's reach throughout your organization.

The USB threat surface

Malicious USB devices can inject commands and exfiltrate data within seconds of being plugged in. USB connections can serve as input, storage, or network devices. The attack surface is large, and threats are fast, quiet, and often go unmonitored.

The MITRE ATT&CK framework classifies these as techniques from removable media,[11] including HID injection via keyboard hijacking, network adapter impersonation that can re-route traffic through malicious networks, and firmware attacks targeting unpatched drivers.

USB enforcement is possible on Linux. Tools like USBGuard allow administrators to block unauthorized devices based on vendor ID, product ID, or device class. But like other Linux security controls, the range of features is not the issue. Keeping track of device state to enable consistent policy enforcement is.

Unlike macOS and Windows, where policies can be applied via MDM controls, Linux administrators need to:

- Deploy USB monitoring and enforcement tools across the entire fleet of Linux workstations.
- Apply USB usage policies. Some enterprises restrict high-risk device classes like storage and unknown HIDs while permitting known-good keyboards, mice, docking stations, and monitors.
- Track changes or deviations from policy. Developers and engineers with root privileges can often bypass USB controls either intentionally or by error.
- Monitor and log usage for threat detection and auditability. Log data is critical for forensic anomaly and threat analysis and enables integration with SIEM and other security tools.

Bluetooth and wireless connections

The threat surface extends beyond ports to wireless connections, particularly Bluetooth. While Wi-Fi is typically managed via 802.1X protocols with certificate-based authentication, Bluetooth often remains a wide-open vector.

Unrestricted Bluetooth connections pose several threats:

- **HID injection** - A malicious Bluetooth device impersonates a keyboard or mouse. Once paired, it can inject keystrokes, open terminals, and execute commands as root.
- **Data exfiltration** - Bluesnarfing enables an attacker to gain unauthorized access to on-device data, including calendars, contacts, and files.
- **Network compromise** - Bluetooth can create a Personal Area Network. If left unsecured, an attacker can connect to a device as a network interface, bypassing network-level firewalls.
- **Service exploitation** - Unpatched Bluetooth software can contain vulnerabilities that allow a proximate attacker to execute arbitrary code without requiring the target device to be discoverable.

Bluetooth controls on Linux are often less mature than on Windows or macOS. Best practices include defaulting to Bluetooth off in high-security environments, restricting which device classes or specific MAC addresses can pair, and logging all pairing events and connection attempts.

The sudo problem

Developers love `sudo`. Enterprises struggle to control it. Privileged access on Linux is treated as a local convenience until unrestricted access leads to a misconfiguration or a breach.

End-users often gain persistent sudo access through group membership. This makes it easy for security teams to lose track of who has root access across a distributed fleet. If users have sudo, any code execution on those devices has a clear escalation path: malware can piggyback on legitimate sudo prompts, attackers can harvest the sudo password, and root access enables persistent backdoors through systemd services or shell initialization changes.

This does not mean sudo should be removed. It means sudo should be treated as a managed capability with lifecycle, review, and logs.

A practical target state includes four elements:

1. **Central source of truth** - Administrators can say “these people have sudo” based on corporate identity, not based on who was added to an unmanaged local device group.
2. **Fast revocation** - Privileged access can be removed quickly when someone leaves, even if a Linux laptop is offline.
3. **Reasonable friction** - If enforcement makes day-to-day work painful, end-users will bypass it.
4. **Auditability** - Logs should show what ran under sudo, when, and which user account initiated it.

Management tools for sudo exist. SSSD can centralize authentication against LDAP or Active Directory. PAM can enforce password policies and MFA. Common patterns for developer workstations include time-bounded sudo elevation, separate admin accounts for privileged actions, command restrictions via the sudoers file, and centralized sudo logging.

Without a centralized management layer, each workstation remains an island with its own local accounts, sudoers file, and audit trail. An attacker will pick the easy target.

Remote lock and wipe

Unlike macOS and Windows, Linux has no native MDM protocol for remote lock and wipe. Many organizations have responded by implementing various controls to prevent unauthorized access to Linux devices: revoking credentials, enabling disk encryption, and moving on.

Disk encryption only protects data at rest, not access to resources via login. Modern Linux management solutions now support remote lock-and-wipe, filling this gap.

What remote lock and wipe protect

Remote actions serve as containment when a loss-of-control incident begins:

- **Remote lock** - When a device is missing but loss of control has not been confirmed, locking provides time to investigate. If the device is online, a lock can cut off access to locally cached credentials, browser sessions, and long-lived tokens.
- **Remote wipe** - Wipe is a clean break. When a device is confirmed lost or compromised, wiping removes local repositories, SSH keys, password managers, and anything in /tmp from the last user session.

Lock and wipe commands only work when a device is online. For offline devices, a parallel path is needed: disabling SSO accounts, revoking SSH keys from authorized stores, and rotating any long-lived tokens. When a device eventually reconnects, remote wipe completes the cleanup.

Test lock-and-wipe actions before deploying them in a crisis. Know your device behavior before you need it.

8. Controlling your software and your data

The SolarWinds attack, uncovered in December 2020, showed what happens when a trusted vendor becomes the attack vector. Malicious code was inserted into legitimate, signed software updates. Thousands of organizations, including U.S. government agencies, deployed those updates in accordance with standard best practices. Every control worked as designed. The software was still compromised.[12]

That incident, and others like it, changed how organizations think about the software running on their devices and the data those devices collect. Two distinct but related concerns now sit at the center of the conversation: software sovereignty and data sovereignty. Both matter for Linux device management. Understanding the difference helps IT leaders make better platform decisions.

Software sovereignty: trusting what runs on your devices

Software sovereignty is about having transparency into, control over, and confidence in the software stack your organization depends on. Can you inspect the code? Can you verify what it does? Can you patch it on your own timeline, or are you waiting for a vendor to act? Can you leave without losing access to your own data?

With proprietary software, the answer to most of those questions is no. Proprietary operating systems and management tools are black boxes. When a vulnerability surfaces, organizations often have to wait for the vendor to acknowledge it, patch it, and release a fix. The organization cannot independently verify the fix or accelerate the timeline.

Open source changes this dynamic. Teams can audit source code, verify behavior, and identify vulnerabilities independently. If a critical flaw needs patching before an upstream fix arrives, it can be patched internally or by forking the project. Community involvement adds another layer of defense. When organizations rely on widely adopted projects like the Linux kernel, they benefit from large numbers of contributors and reviewers examining changes.

Transparency enables detection, but it is not enough on its own

In 2021, a critical remote code execution vulnerability known as Log4Shell[13] was discovered in Apache Log4j, a widely used open-source Java logging library. The flaw had been present in the codebase since 2013. Because Log4j is open source, researchers could immediately inspect the vulnerable code, understand how the attack worked, and share detection tooling publicly within days.

That kind of rapid, distributed response is harder to achieve when the affected software is closed source and only the vendor can inspect or patch it. But the incident also illustrates that openness alone is not sufficient. It highlighted ecosystem issues: under-resourced maintainers, uneven security practices across projects, and the difficulty of coordinating response across a sprawling dependency tree.

Open source provides the conditions for detection. Realizing that benefit requires active investment in review processes and contributor trust models.

Data sovereignty: controlling where your data lives

Data sovereignty is a separate concern from software sovereignty. It is about ensuring that data is subject to the laws and governance structures of the country or region where it is collected or stored. The core question is jurisdictional: who has legal authority over your data, and where does it physically reside?

For device management, this applies directly to the telemetry collected from every computer in your fleet: installed software, configuration state, user identity, policy compliance status, and security events. That data has to go somewhere. Where it goes determines which laws govern it.

With the emergence of cloud computing, that control is no longer guaranteed by physical presence. Companies use vendor-managed platforms across multiple legal jurisdictions. What happens if a national government mandates access to data held by a software vendor? Regulations like GDPR in the EU, data residency requirements in financial services, and frameworks like FedRAMP in the U.S. government all impose constraints on where device data can live and who can access it.

A cloud-only device management platform hosted in a jurisdiction your organization does not control may not meet these requirements. That is not a theoretical risk. It is a procurement constraint that IT leaders in regulated industries encounter regularly.

How Linux and open source address both

Linux and open-source management tools address software sovereignty and data sovereignty at two layers: the operating system and the management platform.

At the operating system layer

Linux is free of the telemetry concerns that affect proprietary operating systems. Windows, for example, collects diagnostic data and transmits it to Microsoft's cloud infrastructure. Organizations can configure the level of telemetry, but cannot eliminate it entirely or independently verify what is sent. Linux does not phone home. The OS does not send telemetry to a vendor's servers unless the administrator explicitly configures it to do so.

From a software-sovereignty perspective, the OS's source code is publicly available. Organizations can inspect it, modify it, and build from it. No single vendor controls access to the operating system or its dependencies.

At the management platform layer

A proprietary, cloud-only management platform collects device data and stores it in the vendor's infrastructure. The organization trusts the vendor to handle that data appropriately, but cannot independently verify the claim.

An open-source management platform changes this in three ways:

- **Inspectable data collection** - The source code that defines what data is collected, how it is transmitted, and where it is stored is publicly available. Teams can audit it. This addresses software sovereignty.
- **Deployment flexibility** - Self-hosted deployment means the organization controls the infrastructure. Device data stays within your network, your cloud account, or your chosen jurisdiction. No vendor has access unless you grant it. This addresses data sovereignty.
- **Portability** - Open-source tools allow data export. If you decide to move to a different platform, your data is not locked inside a vendor's proprietary system. This addresses both.

For organizations subject to GDPR, HIPAA, or government security frameworks, these are not optional features. The ability to host your own management infrastructure, in your own cloud account, in a jurisdiction you choose, is a direct answer to data residency obligations. The ability to inspect the management agent's source code directly addresses software trust requirements.

Linux follows the enterprise trend

Linux on the workstation is not a departure from enterprise norms. It is a continuation of choices most organizations have already made.

Many organizations depend on open-source projects for core services: Nginx and Apache serve a large share of the world's web traffic. Chromium underpins Chrome, Edge, and Brave. OpenSSL secures encrypted connections across the internet. PostgreSQL, MySQL, and SQLite power applications and enterprise databases. Kubernetes orchestrates container workloads at scale.

When teams choose Linux workstations for developers, the local environment can better align with what is deployed to production. If servers, containers, and CI/CD pipelines run Linux, developer workstations that match reduce friction between development and deployment.

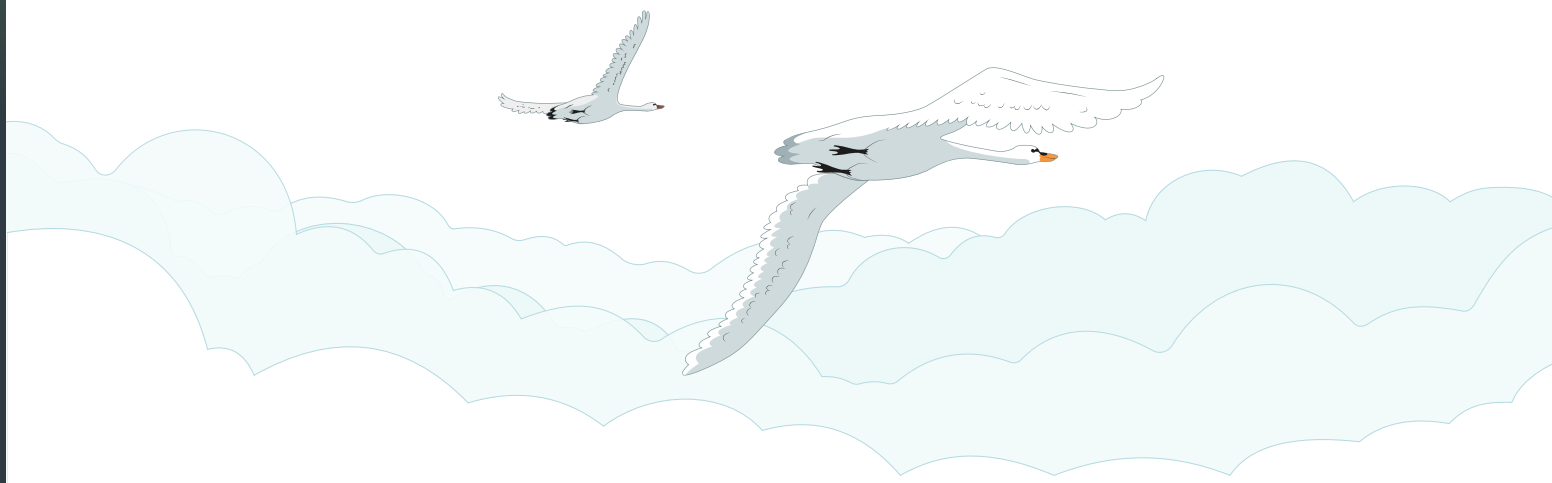
The broader pattern is worth noting: open source already underpins critical workloads across the stack. When that pattern extends to the desktop, Linux adoption looks less like a departure and more like a continuation.

Managing Linux with Linux values

Linux is often chosen for openness and transparency. For organizations that value those qualities, it is worth considering whether the management tooling applied to Linux devices reflects the same principles.

Proprietary management tools can create the same black-box problem that Linux avoids at the operating system level. Teams gain visibility into the OS through open source, but may lose it behind a closed management layer. They cannot inspect how devices are monitored, verify what data is collected, or modify workflows to fit their environment.

Open-source management tools shift this equation. Teams can inspect monitoring logic, verify data collection practices, extend functionality for specific needs, and export data if they decide to move to a different solution.



9. Choosing the right solution

The previous chapters covered why Linux management matters, how to define your requirements, the technical challenges across provisioning, security, software, and device protection, and why controlling both your software stack and your device data is a platform decision. This chapter puts that work to use.

A platform evaluation without defined requirements is a product demo. Evaluate with clear criteria instead of letting a vendor's feature list define your needs.

Business requirements recap

Before comparing platforms, confirm that your shortlist addresses these organizational requirements:

- **Reduce tool sprawl and operational costs** - Each tool carries its own licensing cost, training overhead, and renewal cycle. Consolidating to a single platform reduces that overhead and gives your team a consistent operational model across macOS, Windows, and Linux.
- **Close the Linux compliance gap** - Regulatory frameworks do not grant exemptions by operating system. Unmanaged Linux devices are a gap in your audit trail.
- **Earn trust with technical users** - Linux users resist opaque, intrusive management tooling. The wrong approach drives your best engineers toward workarounds.
- **Control your software and your data** - Trust your tools, control your data. Software sovereignty means auditable code; data sovereignty means choosing where telemetry lives.
- **Lower total cost of ownership** - Understand how platform pricing scales with your fleet before you commit. A platform that fits at 200 devices may become a significant line item at 2,000.
- **Align IT operations with engineering culture** - A platform that supports GitOps-driven workflows reduces the gap between IT and engineering.

Technical requirements recap

Any platform you evaluate should meet these technical requirements:

- **Visibility and auditing** - You must be able to answer basic questions about every Linux device: kernel versions, installed software, configuration state, and patch status, on demand, not just at scheduled intervals.
- **Security baselines and compliance** - The platform must enforce security configurations across Linux, macOS, and Windows, including CIS benchmarks, disk encryption, password policy enforcement, and user management.

- **Open source and ecosystem compatibility** - Linux users are unlikely to trust closed-source, proprietary management tooling. An open-source platform can be inspected, extended, and audited.
- **Deployment flexibility and data sovereignty** - If your organization has data residency requirements, you need a platform that supports self-hosted deployment or region-specific cloud infrastructure.
- **Identity provider integration** - Enrollment, user mapping, software access, and offboarding must integrate with your IdP.
- **Agent performance** - Linux users run resource-intensive workloads. An agent that consumes meaningful CPU or memory will generate complaints and resistance.
- **Security toolchain integration** - Device visibility is only useful if that data reaches the people and systems that act on it.
- **Role-based access control** - IT teams are not monolithic. Security analysts, helpdesk staff, and regional administrators all have different access needs.

Evaluation criteria

Use this table as a conversation guide with vendors. The questions in the “What to verify” column are worth asking directly, not just reading from a datasheet.

Criteria	What to verify
Multi-platform support	Feature parity for disk encryption, software inventory, and compliance reporting across Linux, macOS, and Windows. Linux should not be a degraded experience in the platform's own UI.
Linux as a first-class citizen	Depth of Linux-specific capabilities: configuration enforcement, software management, security baselines, drift detection. Ask what percentage of the vendor's customer base actively manages Linux devices.
Open source	Source code publicly available. Agent behavior can be inspected and verified. Ask whether the platform is built on open-source components or simply has a public GitHub repository.
Ecosystem compatibility	Works with Ansible, Puppet, Salt, and existing scripts. Does not require replacing current automation.
GitOps and workflow flexibility	Supports graphical, command-line, and GitOps workflows without mandating any one approach. Teams targeting Level 3 or Level 4 should treat GitOps support as a firm requirement.

Criteria	What to verify
Deployment flexibility	Supports self-hosted and cloud deployment without treating one as a degraded option. Cloud-agnostic hosting available for organizations with data residency requirements.
Identity provider integration	Integrates with your IdP: Okta, Active Directory, LDAP, or SCIM-compatible systems. Enrollment and offboarding should be automated, not manual.
Agent performance	Agent resource benchmarks available under realistic workloads. Test on hardware representative of your user base before committing.
Security toolchain integration	Integrates with your SIEM, ticketing system, and alerting toolchain. Device telemetry should feed into existing workflows, not require a parallel process.
Compliance framework support	Pre-built CIS benchmark policies available. Disk encryption, password, and user management on Linux. SOC 2 Type II certification for cloud deployments. Ask specifically which version: Type I and Type II are meaningfully different.
Role-based access control	Granular access controls beyond administrator versus read-only. Audit logging of who accessed device data and when, if required by your compliance framework.
Enrollment and migration	Clear enrollment path for existing devices. Self-service, IT-assisted, or fully automated options available. Ask specifically about migration from existing tools or scripts.
Pricing at scale	Understand per-device pricing at your current fleet size and at 2x and 5x growth. Ask which features are gated behind higher tiers.
Active development and vendor stability	Regular releases with Linux functionality shipping alongside macOS and Windows. Public changelog. Defined SLAs, responsive support, and a track record that suggests the vendor will still exist in three years.

Closing thoughts

The Linux MDM market is still maturing. That means more choice, but also more variance in what “Linux support” actually means across vendors. A platform that mentions Linux on its marketing page is not the same as a platform that treats Linux as a first-class concern.

Compliance frameworks do not grant exemptions. Security audits do not skip Linux workstations. Your engineers notice, and resist, tooling that does not respect how they work.

Use the maturity model from Chapter 3 as your target, the requirements from this chapter as your filter, and the evaluation table above as your scorecard. That combination gives you a structured, defensible way to make a platform decision and explain it to your team.

Next steps

Fleet is an open-source device management platform built on osquery. It supports Linux, macOS, Windows, iOS, iPadOS, and Android from a single console. If the criteria in this guide match your requirements:

- Learn more about Linux Device Management at https://fleetdm.com/linux-management
- Learn more about Fleet at https://fleetdm.com
- Request a live demo at https://fleetdm.com/contact

Sources

1. The Register, “Ballmer: Linux is a cancer,” June 2, 2001. https://www.theregister.com/2001/06/02/ballmer_linux_is_a_cancer
2. StatCounter, “Desktop Operating System Market Share United States of America,” December 2025. https://gs.statcounter.com/os-market-share/desktop/united-states-of-america/#monthly-201512-202512
3. Stack Overflow, “Stack Overflow Developer Survey 2025.” https://survey.stackoverflow.co/2025/technology#most-popular-technologies
4. SentinelOne, “IceFire Ransomware Returns | New Linux Samples Suggest a Wider Targeting Scope,” 2023. [URL to be verified before publication] https://www.sentinelone.com/labs/icefire-linux-ransomware/
5. Trend Micro, “The Linux Threat Landscape Report,” 2023. https://www.trendmicro.com/vinfo/us/security/news/cybercrime-and-digital-threats/the-linux-threat-landscape-report
6. Forrester, “The State of Tech Sprawl in the US, 2024.” https://www.forrester.com/report/the-state-of-tech-sprawl-in-the-us-2024/RES181386
7. Kolide / 1Password, “The Shadow IT Report,” September 2023. https://blog.1password.com/unmanaged-devices-run-rampant/
8. Gartner, “Gartner Unveils Top Eight Cybersecurity Predictions for 2023–2024,” March 28, 2023. https://www.gartner.com/en/newsroom/press-releases/2023-03-28-gartner-unveils-top-8-cybersecurity-predictions-for-2023-2024.html
9. Fleet, “Remediating the XZ Vulnerability with Fleet.” https://fleetdm.com/guides/remediating-the-xz-vulnerability-with-fleet
10. CA/Browser Forum, “Ballot SC081v3: Introduce Schedule of Reducing Validity and Data Reuse Periods,” April 2025. https://cabforum.org/2025/04/11/ballot-sc081v3-introduce-schedule-of-reducing-validity-and-data-reuse-periods/
11. MITRE ATT&CK, Technique T1025: Data from Removable Media. https://attack.mitre.org/techniques/T1025/
12. CISA, “Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations,” AA20-352A, December 2020. https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a
13. CISA, “Apache Log4j Vulnerability Guidance,” AA21-356A, December 2021. https://www.cisa.gov/news-events/cybersecurity-advisories/aa21-356a